



Sucher

A standalone device search – files, contacts and calendar events in one field. This guide covers setup and use.

Version 0.29 · GNU GPLv3 · split off from EdgeTab in 2026

Source & APK: github.com/8818freak/Sucher

All screenshots are from real use – personal content has been redacted.

1 What is Sucher?

Sucher searches file *names* AND file *content* (text, Office, PDF, e-books, comic metadata) as well as contacts and calendar events – all in one field. Unlike an edge panel it isn't an always-on service: open, search, done.

Free, open source (GNU GPLv3), no ads, no analytics, no internet permission. Runs on any Android device from version 10.

2 Setup

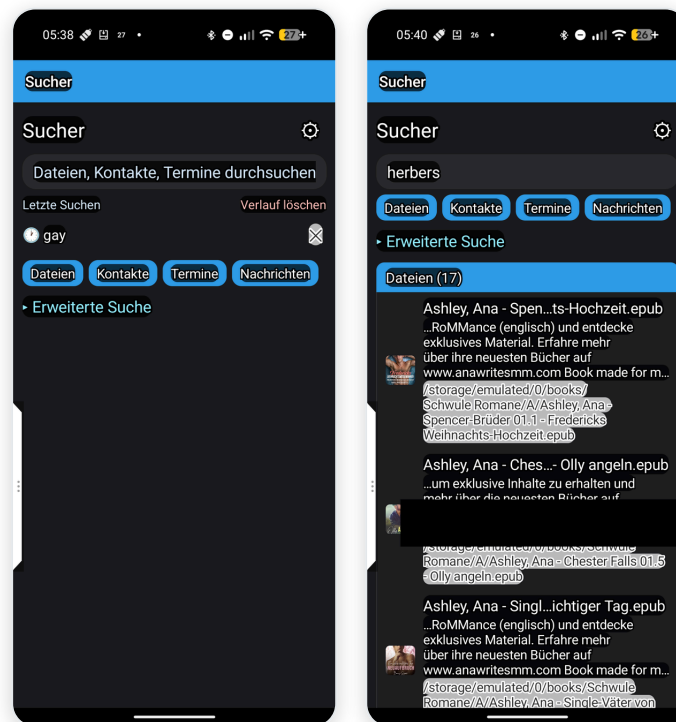
First grant file access (its own system screen) and choose at least one folder. Easiest: the **“Search internal storage”** button adds the whole internal storage directly; **“+ Add another folder”** lets you pick more. Per folder you can enable **“Make content searchable”** – then the full text is indexed in addition to the name (more storage, takes longer).

3 Searching

Type into the field. Category chips narrow the source: *Files* (documents), *Images*, *Videos*, *Music*, *Contacts*, *Events* and *Messages* – each kind gets its own result card. Full-text hits show the matching passage as a snippet; images/PDF/e-books show a thumbnail (generated on demand when displayed). Tapping the thumbnail opens the large preview.

- **✕ in the search field:** clears the query text.
- **New search:** resets everything (text, scopes, advanced fields, filters).
- **📁 Search in folder...:** limits the search to one folder and its subfolders. Important: navigate to the folder first, then tap **“Search in this folder”** – a blue banner shows the active limit and clears it on request.
- **Recent searches:** a collapsible menu under the search field (while it is empty) lists the last search terms; tapping searches again, **“Clear history”** empties the list.

There is **no artificial result cap**: the card header states the full hit count and rows load incrementally via **“Show more”** (in steps of 50). For very many hits, use **“Search in folder”**, the file type, or **“search only within these hits”**.

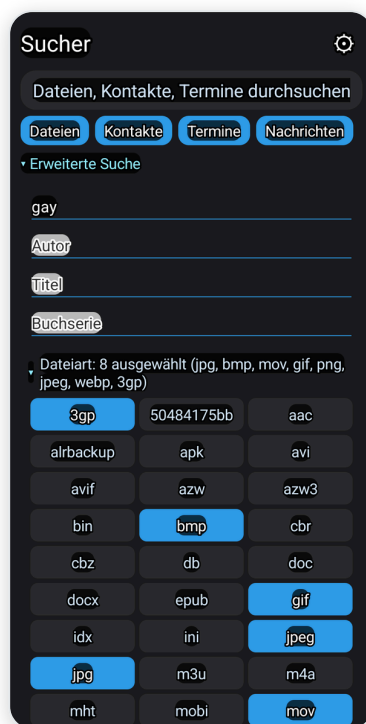


Search home screen

Full-text hits with snippet

4 Advanced search

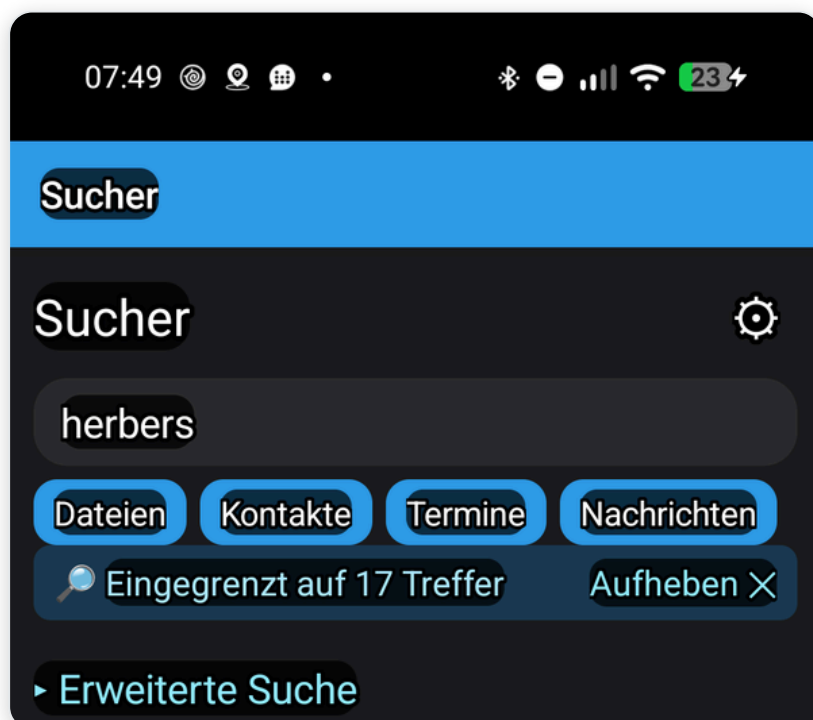
Via “Advanced search”, filter specifically by **author**, **title**, **book series**, **file type** and **date** – ideal for large book and document collections.



Author/title/series/type/date

5 Narrowing results

After any search, “🔍 **search only within these N hits**” restricts every following search to exactly that result set – repeatable, until cleared. A broad first search can thus be refined step by step instead of building one giant combined query.



Narrowing the result set

6 Index & automatic refresh

The index remembers each file's modification time and size and skips everything unchanged on the next run – so follow-up runs are fast. “Reindex now” starts a run by hand; “Automatically refresh in the background every few hours” keeps the index fresh. “Quit Sucher” stops a running index immediately. The progress line names the current phase.

Indexing runs in **three phases**: **Phase 1** captures only metadata (name, type, size, date) of *all* files very quickly – so every file is instantly findable by name, including those in large image, music and video folders. **Phase 2** then fetches full text (for folders with “make content searchable”) and book metadata in batches; **Phase 3** builds thumbnails in batches (they are also created on demand when displayed). This way the expensive full-text work on a large book folder no longer blocks capturing the other folders.

“Reindex” – does it wipe or extend the index?

It **extends/updates** it; it does not wipe it. A run walks the chosen folders and compares each file's modification time with the index: unchanged files are skipped, changed files are re-read, new files are added. Only files that no longer exist (deleted or moved) are removed from the index. An interrupted run therefore never leaves a broken state – just a subset the next run finishes.

What if a file is replaced by one with the same name?

It gets re-indexed. Sucher detects the change by **modification time OR size** – so a replacement is noticed even if it keeps the old modification time (as some tools do) but the size differs. On the next run the metadata and – if content indexing is enabled for that folder – the full text are re-read; the old content is replaced.

Why does indexing sometimes stop and restart?

That is intended and harmless. To save battery, Android gives each background pass only a limited time window; some devices end it after just seconds. Thanks to the fast **Phase 1** (metadata only) even a short window makes big progress, and each pass continues, by modification time/size, exactly where the last left off – nothing is lost, nothing is done twice. The compute-heavy full text (Phase 2) and thumbnails (Phase 3) are caught up gradually. For an immediate full pass, tap “**Reindex now**” and leave the app open (the foreground run has no time limit). Once the index is complete, later runs are short.

If a chosen folder isn't listable (e.g. “/storage/emulated” itself – only “/storage/emulated/0” is accessible), it is automatically mapped to the internal storage and the index is left untouched. A note about it appears in the diagnostics log.

7 Backup & restore

Settings AND the complete search index (all full text already captured) can be backed up to a file and restored from it – handy before switching devices, so the time-consuming indexing isn't needed again.

Thumbnails are deliberately *not* part of the backup (this keeps the file small; they are regenerated on demand). The “**Keep thumbnails permanently**” switch (settings) stores them in app-internal storage – then they survive “clear cache” (including tools like SD Maid); switched off they live in the cache and may be reclaimed under storage pressure.

8 Diagnostics

In the settings, a diagnostics log shows what the indexer did last and where it currently is (folder/file, counters). That makes it possible to see, without a cable, if something ever gets stuck; a “Problematic files” list names files that triggered a hang (and are therefore indexed by name only).